

Boundary Case: Substrate Concurrent-Write Race as Standalone Architectural Treatment in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the **substrate concurrent-write race** as a standalone architectural treatment — articulating how CKS handles multiple cells attempting concurrent writes to overlapping substrate state through human-authored orchestration rules specifying concurrency behavior, rather than through automatic last-write-wins, vendor-managed concurrency control, LLM-mediated arbitration, or any other automatic resolution.

Abstract

A substrate concurrent-write race occurs when two or more cells execute simultaneously and attempt to write to overlapping substrate state. Concurrent writes are operationally common in any deployment running more than one cell, and the treatment is non-obvious because automatic concurrency control — locking, optimistic concurrency, consensus — is the default in most distributed-systems literature. CKS handles the situation differently: concurrent writes are governed by human-authored orchestration rules specifying the concurrency behavior the substrate enforces. Legitimate patterns include rule-specified serialization, conflict registration, substrate partitioning, and optimistic concurrency with rule-specified resolution. Each preserves the determinism contract under timing as bounded non-determinism, distinguishes concurrent writes by cell-execution-id provenance, and keeps conflict-handling determinism intact. Canonical violations include automatic merge, last-write-wins without rule, vendor-managed concurrency, LLM-mediated arbitration, silent dropping, and writing concurrently without any rule specifying behavior. This note formalizes the boundary as standalone, names the architectural commitments it stresses, articulates legitimate and anti-pattern treatments, and bounds the scope against adjacent boundary cases.

1. Why the substrate concurrent-write race needs to be formalized as standalone

The treatment deserves standalone formalization for three reasons: concurrent writes are operationally common rather than exotic, the treatment is non-obvious against the surrounding literature's defaults, and the failure mode it prevents is one the source paper specifically rules out at the substrate level.

Any deployment running more than one cell will, with non-zero probability, encounter a moment at which two cells attempt to write to overlapping substrate state. The probability rises with cell count, scheduling rate, and the fraction of substrate state more than one cell can touch. The boundary is not an edge surfacing under stress; it is a routine condition any multi-cell deployment must handle architecturally.

The default move in distributed-systems literature is to delegate concurrency to an automatic mechanism — last-write-wins, optimistic concurrency with retry, pessimistic locking, consensus across replicas, vendor-managed conflict-resolution policies. None places resolution logic under human authorship as an architectural property. A reader familiar with distributed-systems concurrency control may reasonably assume CKS adopts one of these by default; it does not, and naming the difference precisely is what this note exists to do.

The failure mode the treatment prevents is contradiction-collapse-by-automation, which Claim 3 rules out at the substrate level (§5.3). When concurrent writes are auto-merged or reconciled outside human authorship, the substrate has lost what it committed to carry: the contradiction, addressable as substrate content, governed by rules humans wrote. Load-bearing source-paper sections are §3.3, §5, and §11.3. A6.11 follows A6.01 (rule conflict resolution) and A6.08 (network partition), with which it shares a treatment family.

2. The boundary case scenario

The substrate concurrent-write race is the situation in which two or more cells execute simultaneously and attempt to write to overlapping substrate state. The writes may be intended to coordinate through some logic — two cells whose rules together describe a transactional update — or may be unrelated cells that happen to touch overlapping state by accident of scheduling. From the substrate's perspective, the cause is not architecturally distinguishable; what matters is that more than one write arrives at overlapping state in a window in which neither completes unambiguously before the other begins.

What makes the scenario non-obvious is the combination of three properties. The timing is non-deterministic — order of arrival at the substrate is determined by scheduling, network latency, and other factors not under rule control. The writes may individually be correct under the rules that authorized them, so neither is a candidate for outright rejection. And the substrate's commitment to deterministic state behavior (§11.3) must hold: replay of the same inputs, with timing recorded as bounded non-determinism, must produce the same substrate state.

The architectural question is therefore not "how do we resolve concurrent writes" but "where does the authority over the resolution live, and what does it commit to producing." The answer matches the source paper's general conflict-layer answer: authority lives in human-authored orchestration rules committing to deterministic outcomes under recorded timing.

3. The architectural commitments the boundary case stresses

The **determinism contract** is stressed because timing — the order in which concurrent writes reach the substrate — is non-deterministic. The contract permits bounded non-determinism specifically in the timing category: timing variability is recorded as part of cell-execution-id and substrate-write provenance, and any rule depending on order must specify how the recorded order maps to a deterministic outcome. **Conflict-handling determinism** extends this: the same writes with the same recorded timing must produce the same outcome on every replay.

Conflict-as-first-class is stressed because concurrent writes to overlapping state are often conflicts in Claim 3's sense — two assertions about the same state, neither dominating on validity grounds, both deserving substrate-level preservation unless the rule specifies otherwise. The architecture must accommodate that a concurrent write may produce substrate-level conflict, and the governing rule must say what to do: register the conflict and route it through A6.01, or specify a serialization that prevents it from forming.

Rule authoring is stressed because the treatment depends on a rule existing that names how concurrency must be handled. A deployment whose cell-write footprints can overlap, but whose rules do not include concurrency handling, has not met the authoring obligation. The rule's specification is itself substrate-resident authoritative content — addressable for inspection, modifiable under authority, version-controlled like any other rule.

Bounded non-determinism is stressed in its timing category: timing is not under rule control, but the rule specifies how the recorded timing maps to deterministic outcomes. The boundary case is therefore not a determinism violation; it is the situation in which the bounded-non-determinism discipline must be exercised exactly as the contract names it.

4. The architectural treatment

Concurrent writes to overlapping substrate state are handled per orchestration rules that specify concurrency behavior. Four legitimate patterns are available; the rules choose among them based on substrate structure and cell coordination requirements.

Rule-specified serialization. The rule names the substrate elements over which writes must be serialized and specifies the coordination mechanism — a queue maintained as substrate state, an atomic operation on a primitive the host environment supplies under tool-agnosticism, a lock pattern recorded in the substrate. Concurrent cells acquire the mechanism in arrival order; the wait is treated as bounded non-determinism, and writes complete one at a time, each recorded with its own cell-execution-id provenance.

Conflict registration. The rule specifies that concurrent writes to the substrate elements in question are accepted as first-class conflicts: both writes are recorded, and a substrate-level conflict object is registered against the affected state. Downstream handling follows A6.01 — deferral, human override, delegated adjudicating cell, or any pattern Claim 3 admits. The pattern fits when the substrate's design legitimately admits competing assertions and resolution belongs at the rule-conflict layer rather than the concurrency layer.

Partitioning. The rule specifies that the substrate is partitioned such that cells whose writes might otherwise collide write to different partitions. Concurrency between cells writing to disjoint partitions is not a race

because the writes do not overlap. The pattern fits when the substrate's structure admits a clean partition by cell scope and cross-partition aggregation is itself rule-governed.

Optimistic concurrency with rule-specified resolution. The rule specifies that writes are attempted with version checks against the substrate state being modified, and that a version-mismatch outcome is registered as a conflict — handled per the rule's specification. Optimistic concurrency in this sense remains under the rule layer's authority because mismatch is registered rather than auto-merged, and the resolution logic is human-authored substrate content. Optimistic concurrency that falls back to last-write-wins or vendor-supplied conflict policies is not in this pattern; it is an anti-pattern (§5).

Across all four patterns, three commitments hold uniformly: each concurrent write is recorded with distinct provenance — the six-field metadata distinguishes cell-execution-id, authoring authority, rule version, timestamp, substrate target, and rationale; path retraceability is preserved across the concurrency window; and conflict-handling determinism holds — same writes with same recorded timing produce the same outcome on every replay.

5. Anti-pattern treatments

Eight anti-patterns are detectable as violations.

Auto-merge-concurrent-writes. Concurrent writes are merged automatically — by the substrate, an LLM operation, or runtime middleware — without a human-authored rule that named the merge as the intended outcome. This is a direct instance of contradiction-collapse-by-automation.

Last-write-wins-without-rule. The later-arriving write silently overwrites the earlier write, with no rule specifying that policy and no provenance distinguishing what was overwritten. The earlier write is lost; the conflict disappears without registration.

Vendor-managed-concurrency-control. Outcomes are determined by a vendor's proprietary consensus algorithm, vendor-managed locking, or vendor conflict-resolution policy operating outside substrate governance. Authority has been delegated to the vendor, and tool-agnosticism is also stressed.

LLM-mediated-concurrent-write-resolution. An LLM is consulted to determine which write should "win" or how the writes should be merged. This is the LLM-as-arbiter anti-pattern: authority over substrate state has been delegated to an LLM operation, in violation of the source paper's commitment that the substrate is authoritative and the LLM operates relative to it (§4.1, §11.3).

Silent-dropping-of-one-write. One concurrent write is silently lost — never recorded, never registered as a conflict, never rejected with a recorded reason. Path retraceability fails.

Vendor-specific-optimistic-concurrency-control. Concurrency control depends on a vendor feature that does not migrate to other tools satisfying tool-agnosticism's three minimal requirements. The treatment is in principle correct, but the tool-agnosticism guarantee breaks.

Concurrency-without-rule-specification. Cells write concurrently to overlapping state without any rule naming how the concurrency should be handled. The substrate-cell boundary's rule-mediated-writes property is violated.

Concurrent-write-as-success-without-recording. Both writes complete and the substrate updates, but no provenance distinguishes them — both attributed to the same cell-execution-id, or none recorded — so the substrate cannot later answer which write happened first and under whose authority.

6. Operational implications

Every multi-cell deployment must include orchestration rules naming concurrency behavior for substrate elements more than one cell can write. The rule may be simple (single partition per cell, no overlap) or rich (full conflict registration with downstream A6.01 handling), but it must exist. Deployments that discover their concurrency rules retroactively have already failed the authoring obligation. The four legitimate patterns are not exclusive — a deployment can use serialization for some elements, partitioning for others, conflict registration where competing assertions are legitimate.

Concurrent writes' provenance must distinguish them by cell-execution-id even when both succeed. Replay must be deterministic given recorded timing — verifiable by the reproducibility test — and concurrent writes producing conflicts must be accommodated rather than auto-resolved, verifiable by the conflict-coexistence test. Outcomes varying across replays of the same recorded timing indicate either a non-deterministic rule or an automatic mechanism outside rule control; either is the architectural failure.

Vendor-portable concurrency patterns are preferred. Locking through substrate primitives, serialization through queues maintained as substrate state, and partitioning through rule-defined boundaries all migrate cleanly under tool-agnosticism. Vendor-specific optimistic concurrency or proprietary consensus may be acceptable inside a single deployment if the rule names them explicitly, but they introduce migration risk the deployment must accept consciously.

7. Limits of the architectural treatment

The treatment applies to concurrent writes within a single substrate. Concurrent writes that arise across a network partition — two replicas accepting writes during a split, reconciling on heal — are governed by A6.08, which has its own treatment.

The treatment does not extend to concurrent writes across composition partners. When two substrates compose under multi-substrate composition requirements and both partners process writes that touch shared coordination state, the concurrency is a property of the composition, not of either substrate alone; the composition's rules specify the cross-substrate behavior.

The treatment does not subsume the cell-timeout boundary (A6.10). A cell that times out mid-write and is succeeded by another cell writing to the same state is not a concurrent-write race — the first cell did not complete, and the question is whether its partial state is committed, rolled back, or registered with timeout provenance.

The treatment does not specify particular concurrency-control products, distributed lock managers, or consensus algorithms. It commits to rule-authored specification of concurrency behavior, recorded provenance, and deterministic conflict handling. Which mechanism the rule names — substrate-resident queue, host-supplied atomic primitive, optimistic-with-version-check — is a deployment decision the architecture authorizes, not one it prescribes.

8. Operational test

A deployment handles the substrate concurrent-write race in the architectural sense if and only if all of the following hold whenever two or more cells attempt concurrent writes to overlapping substrate state:

- 1 An orchestration rule names how concurrency between the cells is handled.
- 2 The handling falls within a legitimate pattern: serialization, conflict registration, partitioning, or optimistic concurrency with rule-specified resolution.
- 3 Each concurrent write is recorded with distinct provenance, including cell-execution-id.
- 4 The handling outcome is deterministic given recorded timing as bounded non-determinism, verifiable by the reproducibility test.
- 5 Concurrent writes producing substrate-level conflicts are accommodated by the conflict-coexistence test rather than auto-resolved.
- 6 No LLM operation, vendor mechanism, or runtime middleware determines the concurrency outcome outside the rule's authority.

A deployment that fails any of (1)–(6) is exhibiting one or more of the §5 anti-patterns, and the failure is architectural rather than incidental.

9. Conclusion

The substrate concurrent-write race is operationally common in any multi-cell deployment, and its architectural treatment in CKS is rule-specified concurrency behavior under human authorship rather than automatic resolution under vendor, LLM, or runtime mechanism. Naming the boundary as standalone is what prevents the slide into automatic concurrency control by default — a slide that would replace the source paper's commitment to substrate-level conflict preservation and human-authored rules with whichever distributed-systems mechanism the host environment supplies. The legitimate patterns admitted are diverse enough to cover the operational variety multi-cell deployments encounter, and uniform enough to share the determinism, provenance, and conflict-handling commitments the architecture requires.

A6.11 follows A6.01 and A6.08 within Phase A6's progression. Subsequent notes address multi-author rule authoring conflict during composition (A6.12), AI training-data inclusion (A6.13), deployment-evolution rule version compatibility (A6.14), and schema evolution / substrate migration (A6.15) closing Phase A6. Subsequent work that adopts the CKS pattern, deploys multi-cell substrates, or argues against the architecture should use "the substrate concurrent-write race boundary" in the sense formalized here. Work that handles concurrent writes through automatic mechanisms outside human authorship is using a different architecture, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Boundary Case: Substrate Concurrent-Write Race as Standalone Architectural Treatment in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.